03 · RAG 与知识检索

技术面试准备材料 · AI 应用 / Agent 开发

> 核心问题：怎么把外部知识可靠地送给模型，以及怎么证明「检索得对、答得对」。

1. 请完整讲一下 RAG 的链路

一条完整的 RAG 链路包括离线索引和在线检索两部分。

离线侧：文档接入 → 解析（PDF、HTML、表格、图片）→ 清洗 → 切分 → 生成 Embedding → 写入向量库，同时保留原文、元数据、权限和版本信息。

在线侧：用户问题 → 查询理解与改写 → 检索（向量、关键词或混合）→ 过滤（租户、权限、时间、类型）→ Rerank → 上下文组装 → 模型生成 → 引用与校验 → 返回。

工程上还要有：缓存、超时与降级、检索结果去重、长文档截断、无结果时的拒答策略，以及评测和反馈闭环。

很多人只关注「向量库 + Prompt」，但真正决定效果的是解析、切分、过滤和评测。检索错了，后面生成再强也没用。

2. 文档切分（Chunking）怎么做？切多大合适？

切分的目标是「一个 Chunk 能独立表达一个完整语义」，而不是机械地按字数切。

我会优先按文档结构切：标题、章节、段落、表格、代码块，尽量保持语义边界；结构不清晰时再用滑动窗口，配合一定的重叠（比如 10%～20%）避免跨段信息丢失。

Chunk 大小没有统一答案：太小会丢上下文，太大会引入噪声、稀释相关性，还会挤占生成上下文。一般从几百 Token 起步，根据文档类型和评测结果调整。技术文档可以小一些，叙述性文档可以大一些。

我还会做两件事：一是给每个 Chunk 加元数据（来源、章节、时间、权限、文档版本），方便过滤和引用；二是对父子结构做「小块检索、大块生成」，用小块提高召回精度，再把所属大块送给模型补充上下文。

切分策略必须用评测验证，不能凭感觉。

3. Embedding 和向量库怎么选？

Embedding 主要看四件事：语义效果、语言支持（中文尤其重要）、维度与成本、是否支持增量更新。选型时不要只看榜单，要在自己的业务语料上做召回评测。

向量库看的是：检索性能、过滤能力、多租户隔离、持久化和运维成本、是否支持混合检索。数据量小的时候，pgvector 就够用，运维简单；数据量大、并发高时再考虑专用向量库。

还要注意几个工程细节：Embedding 模型升级会导致向量空间变化，必须重建索引或做双写；不同模型的向量不能混用；删除和更新要有版本机制，否则会出现「文档删了但向量还在」。

我的原则是：先用简单方案跑通，再用评测数据决定要不要换更复杂的组件。

4. 向量检索、BM25、混合检索怎么选?

它们擅长的东西不一样。

向量检索擅长语义相似，比如用户换了一种说法、问的是概念而不是关键词；但对精确术语、编号、错误码、人名这类内容，反而不如关键词检索。

BM25 擅长精确匹配和低频词，可解释、成本低，但对同义表达不敏感。

混合检索把两者结合，通常效果最稳。常见做法是分别召回 Top-K，再用 RRF（倒数排名融合）或加权分数合并。RRF 的好处是不需要把两边的分数归一化，工程上更稳。

我的做法是：默认混合检索；如果业务里精确查询占比高，提高 BM25 权重；如果全是自然语言问答，可以先用向量检索，再根据评测决定是否加 BM25。

不要在没有评测的情况下迷信某一种检索方式。

5. Rerank 的作用是什么？什么时候需要?

Rerank 是在粗召回之后做一次更精细的相关性排序。粗召回追求「不漏」，通常会取比较多候选；Rerank 用交叉编码器或专门模型，把问题和候选逐对打分，追求「排得准」。

它适合这些场景：候选多、噪声大、问题复杂、对答案精度要求高。代价是延迟和成本增加，因为要对每个候选做一次模型推理。

工程上我会先粗召回 20～50 条，再 Rerank 到 5～10 条送给模型；如果延迟敏感，可以减少候选数或只对 Top-N 做 Rerank。

Rerank 不是万能的：如果粗召回里根本没有正确文档，Rerank 也救不回来。所以召回率和 Rerank 效果要分开评估。

6. 查询改写（Query Rewriting）有哪些做法?

常见的有四类。

第一，补全上下文：多轮对话里，把「它」「这个」这类指代补成完整问题。

第二，同义扩展：把口语化表达扩展成业务术语，或者生成多个查询变体。

第三，拆解复杂问题：把多跳问题拆成子问题，分别检索再合并。

第四，HyDE：先让模型生成一个假设答案，再用假设答案去检索。它适合问题很短、语义稀疏的场景，但可能引入偏差，要谨慎使用。

工程上我会把改写结果和原始查询都保留，做多路召回再融合，而不是只信改写后的版本。改写也要有评测，否则可能把原

本能召回的内容改没了。

另外，改写会增加一次模型调用，成本和延迟要算进去。

7. 多租户和权限过滤怎么做?

权限必须在检索阶段做，不能等生成完再过滤。

我会把租户 ID、用户 ID、角色、文档密级、部门等元数据写进索引，检索时用 metadata filter 做强制过滤，而不是靠模型判断。过滤条件由服务端根据身份注入，不能由用户输入决定。

对于复杂权限，可以做「权限预计算」：在写入时把可访问范围编码进索引，查询时直接匹配，避免运行时做大量 join。

还要注意缓存隔离：语义缓存的 key 必须包含租户和权限范围，否则可能把 A 租户的答案返回给 B 租户。

最后要有越权测试：构造不同角色的查询，验证检索结果和生成结果都不会泄露。权限问题一旦出错，后果比效果差严重得多。

8. 如何评估 RAG 的效果?

我会把评估拆成检索和生成两段，避免「答得不好」时不知道是检索问题还是生成问题。

检索侧看：Recall@K、MRR、NDCG、无结果率、误召回率。核心是「正确文档有没有被召回、排得够不够前」。

生成侧看：答案正确性、 faithfulness（是否忠于检索内容）、引用准确率、关键信息覆盖率、拒答准确率、幻觉率。

工程侧看：检索延迟、生成延迟、Token 成本、缓存命中率。

做法上先建带标注的评测集：问题、标准答案、应召回的文档、不可回答的问题。每次改切分、Embedding、Rerank 或 Prompt 都跑一遍回归。

线上再结合用户反馈、人工抽检和转人工率持续验证。离线评测和线上表现要能对上，否则评测集可能已经失真。

9. 检索不到答案时，模型应该怎么处理?

应该允许并鼓励拒答，而不是硬编。

我会在 Prompt 里明确：只能基于给定资料回答；资料不足时说明缺少什么；不能把常识或猜测包装成事实。同时给模型一个明确的「无答案」输出格式，方便系统识别。

系统侧要有置信度判断：检索分数低、召回文档互相矛盾、关键字段缺失时，直接走拒答或转人工，而不是把问题丢给模型自由发挥。

对于「部分可答」的情况，可以分点回答：哪些有依据、哪些不确定、建议用户补充什么信息。

拒答不是失败，错误回答才是。评估时要把「该拒答时拒答」作为一个指标，而不是只看回答率。

10. 知识更新、删除和失效怎么做?

索引要有版本和生命周期管理。

更新时，我会用文档 `ID + 版本号`做幂等写入，新版本生效后再标记旧版本失效，避免更新过程中出现空窗。删除时不能只删向量，还要删原文、缓存和引用，否则会「答得出但点不开」。

对于有时效性的知识，加时间戳和 `TTL`，检索时优先返回新版本，过期内容降权或过滤。

`Embedding` 模型升级时，向量空间会变，必须重建索引或双写双查，不能直接混用。

还要有变更审计：谁在什么时候更新了什么文档、影响了哪些索引。知识库不是静态文件，运维成本和数据库是一个级别。

11. 长文档、表格、图片怎么处理?

不同模态要分开处理。

长文档先做结构解析，按章节和段落切分，保留标题层级；超长章节再做滑动窗口。可以做「摘要索引 + 原文块」：用摘要帮助召回，用原文块生成答案。

表格不要直接当纯文本切碎。我会把表头、行、列关系保留下来，转成结构化表示或 `Markdown` 表格；检索时按表名、列名和关键字段做匹配，生成时把相关行一起送给模型。

图片先用 `OCR` 或多模态模型转成文字描述，保留图片位置和来源；对图表要描述趋势和关键数值，而不是只识别文字。

工程上要给不同模态打标签，检索时按问题类型路由到对应的索引，避免把图片描述和正文混在一起。

12. RAG、长上下文、微调应该怎么选?

三者解决的不是同一个问题。

`RAG` 解决「知识不在模型里、需要实时更新、需要引用和权限控制」的问题，成本低、更新快，是默认选择。

长上下文解决「一次要看的材料很多、关系需要整体理解」的问题，比如长合同分析、跨文档对比。它省掉了检索环节，但成本高、延迟大，而且上下文太长时模型仍可能忽略中间信息。

微调解决「行为、风格、格式、领域表达」的问题，不是用来灌知识的。用微调记事实，更新成本高，还容易过时。

实际系统里经常组合使用：`RAG` 提供事实，微调对齐行为和格式，长上下文处理少量需要整体理解的任务。

我的判断顺序是：先看能不能用 `RAG` 解决；不能再看长上下文；最后才考虑微调，并且要有评测证明它值得。

高频追问

- 为什么向量检索会「召回但不相关」？怎么缓解?

- Chunk 重叠越大越好吗？代价是什么？
- RRF 为什么比直接加权分数更稳？
- 多路召回结果怎么去重和融合？
- 如何防止知识库文档里的 Prompt Injection？
- 评测集里的「不可回答问题」应该怎么构造？